```
!pip install -q torch torchvision matplotlib numpy Pillow tqdm

import os, time, random
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
if torch.cuda.is_available():
    print(f'GPU: {torch.cuda.get_device_name(0)}')
```

```
Using device: cuda
GPU: Tesla T4
```

```
DATA_DIR = './edges2shoes'

if not os.path.exists(DATA_DIR):
    print('Downloading Edges2Shoes...')
    !wget -q --show-progress -O edges2shoes.tar.gz http://efrosgans.eecs.berkeley.edu/pix2pix/datasets/edges2shoes.tar.gz
    !tar -xzf edges2shoes.tar.gz
    !rm edges2shoes.tar.gz
    print('Done.')
else:
    print('Already exists.')

for split in ['train', 'val']:
    path = os.path.join(DATA_DIR, split)
    if os.path.exists(path):
        print(f'{split}: {len(os.listdir(path))} images')
```

```
Downloading Edges2Shoes...
edges2shoes.tar.gz  100%[===================>]   2.02G  6.27MB/s    in 7m 28s
Done.
train: 49825 images
val: 200 images
```

```
IMG_SIZE    = 256
BATCH_SIZE  = 8
EPOCHS      = 50
LR          = 0.0002
BETA1       = 0.5
LAMBDA_L1   = 100.0
TRAIN_LIMIT = 1000
VAL_LIMIT   = 100

class Edges2ShoesDataset(Dataset):
    def __init__(self, root_dir, split='train', img_size=256, limit=None):
        self.root_dir = os.path.join(root_dir, split)
        self.files = sorted([f for f in os.listdir(self.root_dir)
                             if f.lower().endswith(('.jpg','.png','.jpeg'))])
        if limit:
            self.files = self.files[:limit]
        self.transform = transforms.Compose([
            transforms.Resize((img_size, img_size)),
            transforms.ToTensor(),
            transforms.Normalize([0.5]*3, [0.5]*3)
        ])

    def __len__(self): return len(self.files)

    def __getitem__(self, idx):
        img = Image.open(os.path.join(self.root_dir, self.files[idx])).convert('RGB')
        w, h = img.size; half = w // 2
        edge = img.crop((0, 0, half, h))
        real = img.crop((half, 0, w, h))
        return self.transform(edge), self.transform(real)

train_ds = Edges2ShoesDataset(DATA_DIR, 'train', IMG_SIZE, TRAIN_LIMIT)
val_ds   = Edges2ShoesDataset(DATA_DIR, 'val',   IMG_SIZE, VAL_LIMIT)
```

```
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,  num_workers=2)
val_loader   = DataLoader(val_ds,   batch_size=4,          shuffle=False, num_workers=2)

print(f'Train: {len(train_ds)} | Val: {len(val_ds)}')
```

```
Train: 1000 | Val: 100
```

```
# UNET generator

class UNetDown(nn.Module):
    def __init__(self, in_ch, out_ch, normalize=True, dropout=0.0):
        super().__init__()
        layers = [nn.Conv2d(in_ch, out_ch, 4, stride=2, padding=1, bias=False)]
        if normalize: layers.append(nn.BatchNorm2d(out_ch))
        layers.append(nn.LeakyReLU(0.2, inplace=True))
        if dropout: layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x): return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_ch, out_ch, dropout=0.0):
        super().__init__()
        layers = [
            nn.ConvTranspose2d(in_ch, out_ch, 4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True)
        ]
        if dropout: layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x, skip):
        return torch.cat([self.model(x), skip], dim=1)

class UNetGenerator(nn.Module):
    def __init__(self, in_ch=3, out_ch=3):
        super().__init__()
        self.d1 = UNetDown(in_ch,  64,  normalize=False)
        self.d2 = UNetDown(64,  128)
        self.d3 = UNetDown(128, 256)
        self.d4 = UNetDown(256, 512)
        self.d5 = UNetDown(512, 512)
        self.d6 = UNetDown(512, 512)
        self.d7 = UNetDown(512, 512)
        self.d8 = UNetDown(512, 512, normalize=False)

        self.u1 = UNetUp(512,  512, dropout=0.5)
        self.u2 = UNetUp(1024, 512, dropout=0.5)
        self.u3 = UNetUp(1024, 512, dropout=0.5)
        self.u4 = UNetUp(1024, 512)
        self.u5 = UNetUp(1024, 256)
        self.u6 = UNetUp(512,  128)
        self.u7 = UNetUp(256,   64)
        self.final = nn.Sequential(
            nn.ConvTranspose2d(128, out_ch, 4, stride=2, padding=1), nn.Tanh()
        )

    def forward(self, x):
        d1=self.d1(x); d2=self.d2(d1); d3=self.d3(d2); d4=self.d4(d3)
        d5=self.d5(d4); d6=self.d6(d5); d7=self.d7(d6); d8=self.d8(d7)
        u1=self.u1(d8,d7); u2=self.u2(u1,d6); u3=self.u3(u2,d5); u4=self.u4(u3,d4)
        u5=self.u5(u4,d3); u6=self.u6(u5,d2); u7=self.u7(u6,d1)
        return self.final(u7)

print('U-Net Generator ready.')
```

```
U-Net Generator ready.
```

```
# patchgan discriminator


class PatchGANDiscriminator(nn.Module):
    def __init__(self, in_ch=6):
        super().__init__()
        def block(ic, oc, norm=True):
            layers = [nn.Conv2d(ic, oc, 4, stride=2, padding=1)]
            if norm: layers.append(nn.BatchNorm2d(oc))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *block(in_ch,  64, norm=False),
            *block(64,  128),
            *block(128, 256),
            nn.Conv2d(256, 512, 4, stride=1, padding=1),
```

```
            nn.BatchNorm2d(512), nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(512, 1, 4, stride=1, padding=1)
        )
    def forward(self, condition, image):
        return self.model(torch.cat([condition, image], dim=1))

print('PatchGAN Discriminator ready.')
```

```
PatchGAN Discriminator ready.
```

```
# baseline CNN

class BaselineCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(3,   64,  4, stride=2, padding=1), nn.LeakyReLU(0.2, True),
            nn.Conv2d(64,  128, 4, stride=2, padding=1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2, True),
            nn.Conv2d(128, 256, 4, stride=2, padding=1), nn.BatchNorm2d(256), nn.LeakyReLU(0.2, True),
            nn.Conv2d(256, 512, 4, stride=2, padding=1), nn.BatchNorm2d(512), nn.LeakyReLU(0.2, True),
            nn.Conv2d(512, 512, 4, stride=2, padding=1), nn.BatchNorm2d(512), nn.LeakyReLU(0.2, True),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(512, 512, 4, stride=2, padding=1), nn.BatchNorm2d(512), nn.ReLU(True),
            nn.ConvTranspose2d(512, 256, 4, stride=2, padding=1), nn.BatchNorm2d(256), nn.ReLU(True),
            nn.ConvTranspose2d(256, 128, 4, stride=2, padding=1), nn.BatchNorm2d(128), nn.ReLU(True),
            nn.ConvTranspose2d(128,  64, 4, stride=2, padding=1), nn.BatchNorm2d(64),  nn.ReLU(True),
            nn.ConvTranspose2d( 64,   3, 4, stride=2, padding=1), nn.Tanh()
        )
    def forward(self, x): return self.decoder(self.encoder(x))

print('Baseline CNN ready.')
```

```
Baseline CNN ready.
```

```
# init models and optimizers

def weights_init(m):
    if isinstance(m, (nn.Conv2d, nn.ConvTranspose2d)):
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif isinstance(m, nn.BatchNorm2d):
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

G   = UNetGenerator().to(device);          G.apply(weights_init)
D   = PatchGANDiscriminator().to(device);  D.apply(weights_init)
CNN = BaselineCNN().to(device);            CNN.apply(weights_init)

criterion_GAN = nn.BCEWithLogitsLoss()
criterion_L1  = nn.L1Loss()

opt_G   = optim.Adam(G.parameters(),   lr=LR, betas=(BETA1, 0.999))
opt_D   = optim.Adam(D.parameters(),   lr=LR, betas=(BETA1, 0.999))
opt_CNN = optim.Adam(CNN.parameters(), lr=LR, betas=(BETA1, 0.999))

print(f'G params:   {sum(p.numel() for p in G.parameters()):,}')
print(f'D params:   {sum(p.numel() for p in D.parameters()):,}')
print(f'CNN params: {sum(p.numel() for p in CNN.parameters()):,}')
```

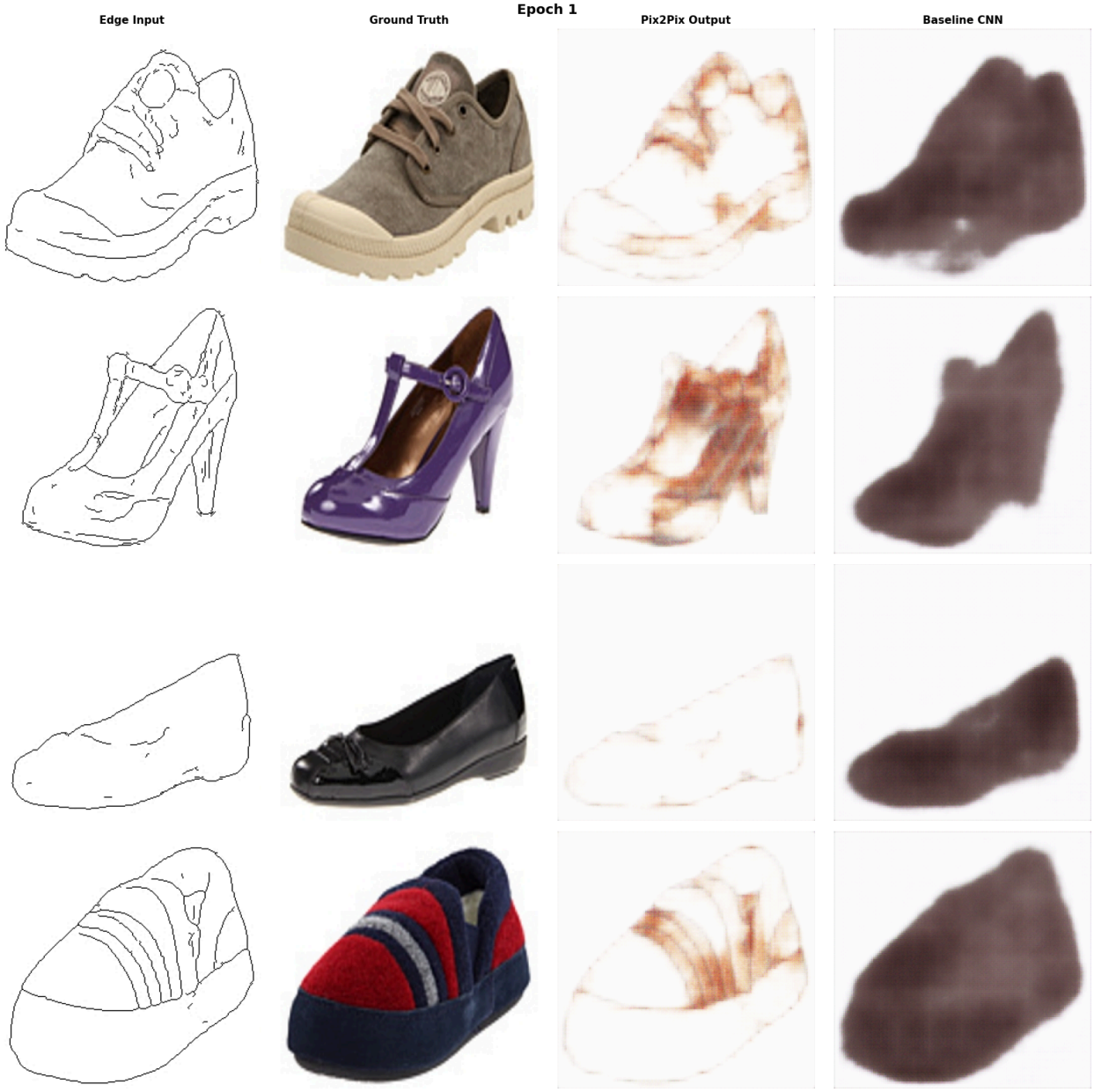
```
G params:   54,413,955
D params:   2,769,601
CNN params: 13,906,947
```

```
# training loop

history = {'G_loss':[], 'D_loss':[], 'G_adv':[], 'G_L1':[], 'CNN_loss':[]}

fixed_edges, fixed_reals = next(iter(val_loader))
fixed_edges = fixed_edges.to(device)
fixed_reals = fixed_reals.to(device)

def denorm(x):
    return (x * 0.5 + 0.5).clamp(0, 1)

def visualise(epoch):
    G.eval(); CNN.eval()
    with torch.no_grad():
        gan_out = G(fixed_edges)
        cnn_out = CNN(fixed_edges)
    fig, axes = plt.subplots(4, 4, figsize=(16, 16))
    for col, (title, imgs) in enumerate(zip(
        ['Edge Input', 'Ground Truth', 'Pix2Pix Output', 'Baseline CNN'],
```

```
        [fixed_edges, fixed_reals, gan_out, cnn_out]
    )):
        for row in range(4):
            img = denorm(imgs[row]).cpu().permute(1,2,0).numpy().clip(0,1)
            axes[row, col].imshow(img)
            if row == 0: axes[row, col].set_title(title, fontsize=11, fontweight='bold')
            axes[row, col].axis('off')
    plt.suptitle(f'Epoch {epoch}', fontsize=14, fontweight='bold')
    plt.tight_layout(); plt.show()
    G.train(); CNN.train()

print('Starting training...')
start = time.time()

for epoch in range(1, EPOCHS + 1):
    G.train(); D.train(); CNN.train()
    tot_G = tot_D = tot_adv = tot_l1 = tot_cnn = 0
    n = len(train_loader)

    for edges, reals in train_loader:
        edges = edges.to(device); reals = reals.to(device)

        real_lbl = torch.ones (D(edges, reals).shape, device=device)
        fake_lbl = torch.zeros(D(edges, reals).shape, device=device)

        # Discriminator
        opt_D.zero_grad()
        fakes = G(edges).detach()
        loss_D = 0.5 * (criterion_GAN(D(edges, reals), real_lbl) +
                        criterion_GAN(D(edges, fakes), fake_lbl))
        loss_D.backward(); opt_D.step()

        # Generator
        opt_G.zero_grad()
        fakes   = G(edges)
        l_adv   = criterion_GAN(D(edges, fakes), real_lbl)
        l_l1    = criterion_L1(fakes, reals) * LAMBDA_L1
        loss_G  = l_adv + l_l1
        loss_G.backward(); opt_G.step()

        # Baseline CNN
        opt_CNN.zero_grad()
        loss_cnn = criterion_L1(CNN(edges), reals)
        loss_cnn.backward(); opt_CNN.step()

        tot_G += loss_G.item(); tot_D += loss_D.item()
        tot_adv += l_adv.item(); tot_l1 += l_l1.item()
        tot_cnn += loss_cnn.item()

    history['G_loss'].append(tot_G/n); history['D_loss'].append(tot_D/n)
    history['G_adv'].append(tot_adv/n); history['G_L1'].append(tot_l1/n)
    history['CNN_loss'].append(tot_cnn/n)

    elapsed = (time.time()-start)/60
    eta     = elapsed/epoch * (EPOCHS-epoch)
    print(f'[{epoch:3d}/{EPOCHS}] G:{tot_G/n:.4f} D:{tot_D/n:.4f} '
          f'Adv:{tot_adv/n:.4f} L1:{tot_l1/n:.2f} CNN:{tot_cnn/n:.4f} '
          f'| {elapsed:.1f}m elapsed, ETA {eta:.1f}m')

    if epoch % 15 == 0 or epoch == 1 or epoch == 50:
        visualise(epoch)

print(f'\nTraining complete in {(time.time()-start)/60:.1f} min')
```

```
Starting training...
[  1/50] G:30.2229 D:0.4281 Adv:1.7898 L1:28.43 CNN:0.3326 | 1.0m elapsed, ETA 50.7m
```

Epoch 1




```
[  2/50] G:22.8902 D:0.3983 Adv:1.8211 L1:21.07 CNN:0.2194 | 2.1m elapsed, ETA 51.6m
[  3/50] G:20.3580 D:0.4225 Adv:1.6732 L1:18.68 CNN:0.1902 | 3.2m elapsed, ETA 50.6m
[  4/50] G:18.3479 D:0.4814 Adv:1.5616 L1:16.79 CNN:0.1621 | 4.3m elapsed, ETA 49.5m
[  5/50] G:17.1725 D:0.5131 Adv:1.3778 L1:15.79 CNN:0.1439 | 5.4m elapsed, ETA 48.5m
[  6/50] G:15.8497 D:0.5487 Adv:1.2411 L1:14.61 CNN:0.1268 | 6.5m elapsed, ETA 47.4m
[  7/50] G:14.9542 D:0.5533 Adv:1.2118 L1:13.74 CNN:0.1171 | 7.5m elapsed, ETA 46.4m
[  8/50] G:14.3461 D:0.5795 Adv:1.1256 L1:13.22 CNN:0.1109 | 8.6m elapsed, ETA 45.3m
[  9/50] G:13.3427 D:0.5717 Adv:1.0597 L1:12.28 CNN:0.1041 | 9.7m elapsed, ETA 44.2m
[ 10/50] G:12.6410 D:0.6030 Adv:1.0798 L1:11.56 CNN:0.0985 | 10.8m elapsed, ETA 43.2m
[ 11/50] G:12.1800 D:0.5995 Adv:1.0547 L1:11.13 CNN:0.0941 | 11.9m elapsed, ETA 42.1m
[ 12/50] G:11.5060 D:0.5885 Adv:1.1270 L1:10.38 CNN:0.0898 | 13.0m elapsed, ETA 41.0m
[ 13/50] G:10.9015 D:0.5703 Adv:1.0973 L1:9.80 CNN:0.0863 | 14.0m elapsed, ETA 39.9m
[ 14/50] G:10.3201 D:0.6045 Adv:1.1063 L1:9.21 CNN:0.0833 | 15.1m elapsed, ETA 38.9m
[ 15/50] G:9.9871 D:0.5992 Adv:1.0781 L1:8.91 CNN:0.0813 | 16.2m elapsed, ETA 37.8m
```

Epoch 15

```
[ 16/50] G:9.7722 D:0.5848 Adv:1.1063 L1:8.67 CNN:0.0795 | 17.3m elapsed, ETA 36.7m
[ 17/50] G:9.4116 D:0.6004 Adv:1.0740 L1:8.34 CNN:0.0770 | 18.4m elapsed, ETA 35.7m
[ 18/50] G:9.1858 D:0.5807 Adv:1.0870 L1:8.10 CNN:0.0751 | 19.4m elapsed, ETA 34.6m
[ 19/50] G:8.9837 D:0.6001 Adv:1.0828 L1:7.90 CNN:0.0735 | 20.5m elapsed, ETA 33.5m
[ 20/50] G:8.7545 D:0.5833 Adv:1.0756 L1:7.68 CNN:0.0712 | 21.6m elapsed, ETA 32.4m
[ 21/50] G:8.5353 D:0.5985 Adv:1.0897 L1:7.45 CNN:0.0685 | 22.7m elapsed, ETA 31.3m
[ 22/50] G:8.4859 D:0.5871 Adv:1.0860 L1:7.40 CNN:0.0669 | 23.8m elapsed, ETA 30.3m
[ 23/50] G:8.3856 D:0.5814 Adv:1.0601 L1:7.33 CNN:0.0649 | 24.9m elapsed, ETA 29.2m
[ 24/50] G:8.0887 D:0.5969 Adv:1.0324 L1:7.06 CNN:0.0633 | 25.9m elapsed, ETA 28.1m
[ 25/50] G:7.8968 D:0.5929 Adv:1.0406 L1:6.86 CNN:0.0611 | 27.0m elapsed, ETA 27.0m
[ 26/50] G:7.7903 D:0.5897 Adv:1.0477 L1:6.74 CNN:0.0596 | 28.1m elapsed, ETA 25.9m
[ 27/50] G:7.6728 D:0.5955 Adv:1.0580 L1:6.61 CNN:0.0583 | 29.2m elapsed, ETA 24.9m
[ 28/50] G:7.6146 D:0.5925 Adv:1.0305 L1:6.58 CNN:0.0571 | 30.3m elapsed, ETA 23.8m
[ 29/50] G:7.4498 D:0.6009 Adv:1.0063 L1:6.44 CNN:0.0558 | 31.3m elapsed, ETA 22.7m
[ 30/50] G:7.2600 D:0.6142 Adv:1.0168 L1:6.24 CNN:0.0551 | 32.4m elapsed, ETA 21.6m
```

Epoch 30

```
[ 31/50] G:7.1778 D:0.6108 Adv:0.9630 L1:6.21 CNN:0.0543 | 33.5m elapsed, ETA 20.5m
[ 32/50] G:7.0821 D:0.6200 Adv:0.9731 L1:6.11 CNN:0.0679 | 34.6m elapsed, ETA 19.5m
[ 33/50] G:7.0016 D:0.6124 Adv:0.9516 L1:6.05 CNN:0.0629 | 35.7m elapsed, ETA 18.4m
[ 34/50] G:6.9007 D:0.6222 Adv:0.9451 L1:5.96 CNN:0.0543 | 36.8m elapsed, ETA 17.3m
[ 35/50] G:6.7340 D:0.6247 Adv:0.9365 L1:5.80 CNN:0.0508 | 37.8m elapsed, ETA 16.2m
[ 36/50] G:6.5993 D:0.6295 Adv:0.9397 L1:5.66 CNN:0.0491 | 38.9m elapsed, ETA 15.1m
[ 37/50] G:6.4764 D:0.6244 Adv:0.9263 L1:5.55 CNN:0.0480 | 40.0m elapsed, ETA 14.1m
[ 38/50] G:6.4328 D:0.6215 Adv:0.9343 L1:5.50 CNN:0.0475 | 41.1m elapsed, ETA 13.0m
[ 39/50] G:6.3313 D:0.6210 Adv:0.9285 L1:5.40 CNN:0.0470 | 42.2m elapsed, ETA 11.9m
[ 40/50] G:6.3208 D:0.6107 Adv:0.9613 L1:5.36 CNN:0.0467 | 43.2m elapsed, ETA 10.8m
[ 41/50] G:6.3053 D:0.6564 Adv:1.0227 L1:5.28 CNN:0.0463 | 44.3m elapsed, ETA 9.7m
[ 42/50] G:6.3413 D:0.5210 Adv:1.1113 L1:5.23 CNN:0.0459 | 45.4m elapsed, ETA 8.6m
[ 43/50] G:6.8455 D:0.3624 Adv:1.5341 L1:5.31 CNN:0.0451 | 46.5m elapsed, ETA 7.6m
[ 44/50] G:7.4820 D:0.2862 Adv:1.9317 L1:5.55 CNN:0.0442 | 47.6m elapsed, ETA 6.5m
[ 45/50] G:7.9685 D:0.4302 Adv:2.1847 L1:5.78 CNN:0.0441 | 48.6m elapsed, ETA 5.4m
```

Epoch 45

Edge Input **Ground Truth** **Pix2Pix Output** **Baseline CNN**

```
[ 46/50] G:7.5556 D:0.4477 Adv:1.8699 L1:5.69 CNN:0.0440 | 49.7m elapsed, ETA 4.3m
[ 47/50] G:7.3396 D:0.4556 Adv:1.8007 L1:5.54 CNN:0.0435 | 50.8m elapsed, ETA 3.2m
[ 48/50] G:7.0118 D:0.4592 Adv:1.6360 L1:5.38 CNN:0.0427 | 51.9m elapsed, ETA 2.2m
[ 49/50] G:6.8057 D:0.4857 Adv:1.5270 L1:5.28 CNN:0.0423 | 53.0m elapsed, ETA 1.1m
[ 50/50] G:6.6986 D:0.4767 Adv:1.5036 L1:5.20 CNN:0.0420 | 54.1m elapsed, ETA 0.0m
```

Epoch 50

Edge Input **Ground Truth** **Pix2Pix Output** **Baseline CNN**